

Foundations & Searching

1. Explain the difference between linear search and binary search. When would you use each?
 - Answer: Linear search checks each element one by one ($O(n)$), making it useful for unsorted data. Binary search ($O(\log n)$) repeatedly divides a sorted dataset in half, making it more efficient when the data is already sorted.
2. What are the advantages of indexing in a database, and what types of indexes exist?
 - Answer: Indexing improves search performance by creating a sorted data structure. Types include B-trees, hash indexes, and full-text indexes.
3. Describe the pros and cons of arrays vs. linked lists in the context of database storage.
 - Answer: Arrays allow fast random access but slow insertions, whereas linked lists allow fast insertions but slow random access.

Binary Search Trees (BST) & AVL Trees

4. How does a binary search tree (BST) ensure efficient searching? Provide an example of an insertion process.
 - Answer: BSTs maintain order by placing smaller values on the left and larger values on the right. Example: Inserting 7 into {10, 5, 15} results in {10, 5, 15, 7}.
5. What is an AVL tree, and how does it maintain balance?
 - Answer: An AVL tree is a self-balancing BST where the height difference between left and right subtrees is at most 1. It uses rotations (LL, RR, LR, RL) to rebalance when necessary.
6. What is the purpose of tree rotations in AVL trees? Provide an example of a left-right (LR) rotation.
 - Answer: Tree rotations restore balance when an insertion causes an imbalance. In an LR rotation, a left rotation is performed on the left child, followed by a right rotation on the root.

B-Trees & B+ Trees

7. Compare B-Trees and B+ Trees. Why might a database use one over the other?

- Answer: B-Trees store keys and values in internal and leaf nodes, making random access efficient. B+ Trees store values only in leaf nodes, making range queries faster.

8. What happens during the insertion process in a B-tree?

- Answer: A key is inserted into a node. If the node exceeds the allowed number of keys, it splits, promoting a key to the parent. If the root splits, a new root is created.

9. Why do B+ Trees improve range queries compared to B-Trees?

- Answer: Leaf nodes in B+ Trees are linked, allowing efficient sequential access without needing to traverse the entire tree.

Relational Databases & ACID Properties

10. Explain the ACID properties of transactions and why they are important.

- Answer:
- Atomicity: Transactions are all-or-nothing.
- Consistency: Database transitions from one valid state to another.
- Isolation: Transactions do not interfere with each other.
- Durability: Changes persist even after a failure.

11. What are common concurrency issues in databases, and how are they addressed?

- Answer: Issues include dirty reads, non-repeatable reads, and phantom reads. Solutions include locking mechanisms and transaction isolation levels.

NoSQL Databases & CAP Theorem

12. Explain the CAP theorem and its trade-offs in distributed databases.

- Answer: CAP states that a distributed database cannot simultaneously achieve Consistency (C), Availability (A), and Partition Tolerance (P).
 - CP (Consistency & Partition Tolerance) sacrifices Availability.
 - AP (Availability & Partition Tolerance) sacrifices Consistency.
 - CA (Consistency & Availability) sacrifices Partition Tolerance.

13. Compare key-value, document, and graph databases. When would you use each?

- Answer:
- Key-value stores (Redis) are best for caching.
- Document databases (MongoDB) work well for semi-structured data.
- Graph databases (Neo4j) are best for complex relationships like social networks.

Replication & Distributed Databases

14. What are the benefits and challenges of data replication?

- Answer: Benefits include fault tolerance, load balancing, and low latency. Challenges include consistency management and handling conflicts.

15. Describe leader-based replication and how it handles write operations.

- Answer: In leader-based replication, a single leader handles writes and propagates changes to followers. Failover mechanisms elect a new leader if the current one fails.

Searching, Indexing, and Data Structures

1. Why is binary search not suitable for unsorted data? How can we improve search efficiency in such cases?

- Answer: Binary search requires sorted data to function. For unsorted data, indexing (e.g., hash indexes or B-Trees) or sorting before searching can improve efficiency.

2. What is the difference between a primary index and a secondary index in a database?

- Answer: A primary index is built on a primary key and enforces uniqueness. A secondary index is created on non-primary key attributes to improve search efficiency.
3. How does hashing work in database search, and why does it provide $O(1)$ time complexity?
- Answer: Hashing uses a hash function to directly map keys to memory locations, enabling near-instant lookups. However, hash collisions can affect performance.

Trees and Balanced Data Structures

4. Describe the process of deleting a node in a Binary Search Tree (BST). How does it differ if the node has zero, one, or two children?
- Answer:
 - If the node has no children, simply remove it.
 - If it has one child, replace it with its child.
 - If it has two children, replace it with the in-order successor and delete that node.
5. Explain the advantages of an AVL tree over a standard BST.
- Answer: AVL trees maintain a balanced height, ensuring $O(\log n)$ search, insertion, and deletion times, whereas an unbalanced BST can degrade to $O(n)$.
6. What are the time complexities of inserting, deleting, and searching in an AVL tree?
- Answer:
 - Insertion: $O(\log n)$ due to rebalancing
 - Deletion: $O(\log n)$ due to rebalancing
 - Search: $O(\log n)$ due to the balanced structure

B-Trees, B+ Trees, and Indexing

7. What is the key difference between B-Trees and AVL trees in terms of database usage?

- Answer: B-Trees are used in databases because they minimize disk I/O by storing multiple keys per node, whereas AVL trees are optimized for in-memory operations.
8. Describe the steps involved in splitting a node in a B-Tree during insertion.
- Answer:
 - Insert the new key in sorted order.
 - If the node exceeds the maximum allowed keys, split it into two nodes.
 - Promote the middle key to the parent node.
 - If the parent also exceeds its key limit, repeat the split.
9. Why are B+ Trees preferred over B-Trees for range queries?
- Answer: B+ Trees store data only in leaf nodes, which are linked sequentially, allowing for efficient range scans without traversing internal nodes.

Transactions and ACID Properties

10. Give an example of a situation where atomicity prevents data corruption in a database transaction.
- Answer: In a banking system, if a transfer transaction deducts money from one account but fails before adding it to another, atomicity ensures a rollback to prevent an inconsistent state.
11. Explain the difference between strict, read committed, and repeatable read isolation levels in a database.
- Answer:
 - Strict isolation: No transactions interfere with each other.
 - Read committed: Only committed changes are visible to other transactions.
 - Repeatable read: Ensures the same data is read within a transaction, preventing non-repeatable reads.

12. How does a database handle conflicts in concurrent transactions? Provide an example.

- Answer: Databases use locking mechanisms (pessimistic concurrency) or versioning (optimistic concurrency). Example: If two users attempt to book the last ticket simultaneously, one transaction succeeds while the other fails or retries.

NoSQL, Distributed Databases, and CAP Theorem

13. Explain how eventual consistency works in NoSQL databases. Give a real-world example.

- Answer: Eventual consistency allows distributed databases to return outdated data temporarily, but ensures updates propagate over time. Example: Amazon DynamoDB may show slightly delayed inventory updates after a sale.

14. Compare leader-based and leaderless replication models in distributed databases.

- Answer:
- Leader-based replication: A single leader handles writes, followers replicate changes.
- Leaderless replication: Any node can accept writes, and conflict resolution occurs later.

15. What are the advantages and disadvantages of using a document-based NoSQL database like MongoDB?

- Answer:
Advantages: Flexible schema, fast reads/writes, and easy horizontal scaling.
Disadvantages: Lack of strict consistency, complex queries can be slower than SQL databases.